College Operations Copilot

AI-Powered College Documentation & Operations Platform

Product	College Operations Copilot — first workflow: Activity Documentation Automation
Document	Architecture document
Version	1.0 (architecture) · software at commit `7119b13`
Date	29 September 2026
Pilot context	Department of Mechanical Engineering, K.D.K. College of Engineering, Nagpur
Nature	Original proposed architecture · implementation status clearly marked

How to read this document. Every component is labelled IMPLEMENTED — present in the code and exercised by automated tests; OPTIONAL — implemented but off by default; or PROPOSED / FUTURE — design only, not built. HUMAN marks points where a person decides.

No student names, signatures, attendance records or private institutional documents are reproduced here. This document makes no claim that individual technologies or concepts are globally unprecedented (see “Originality and Scope”).

1. Problem and product

What the software is. It is a web application for faculty. It turns the evidence of a college activity — the notice or poster, the attendance sheet, photographs and the event details — into the institution's standard documentation:

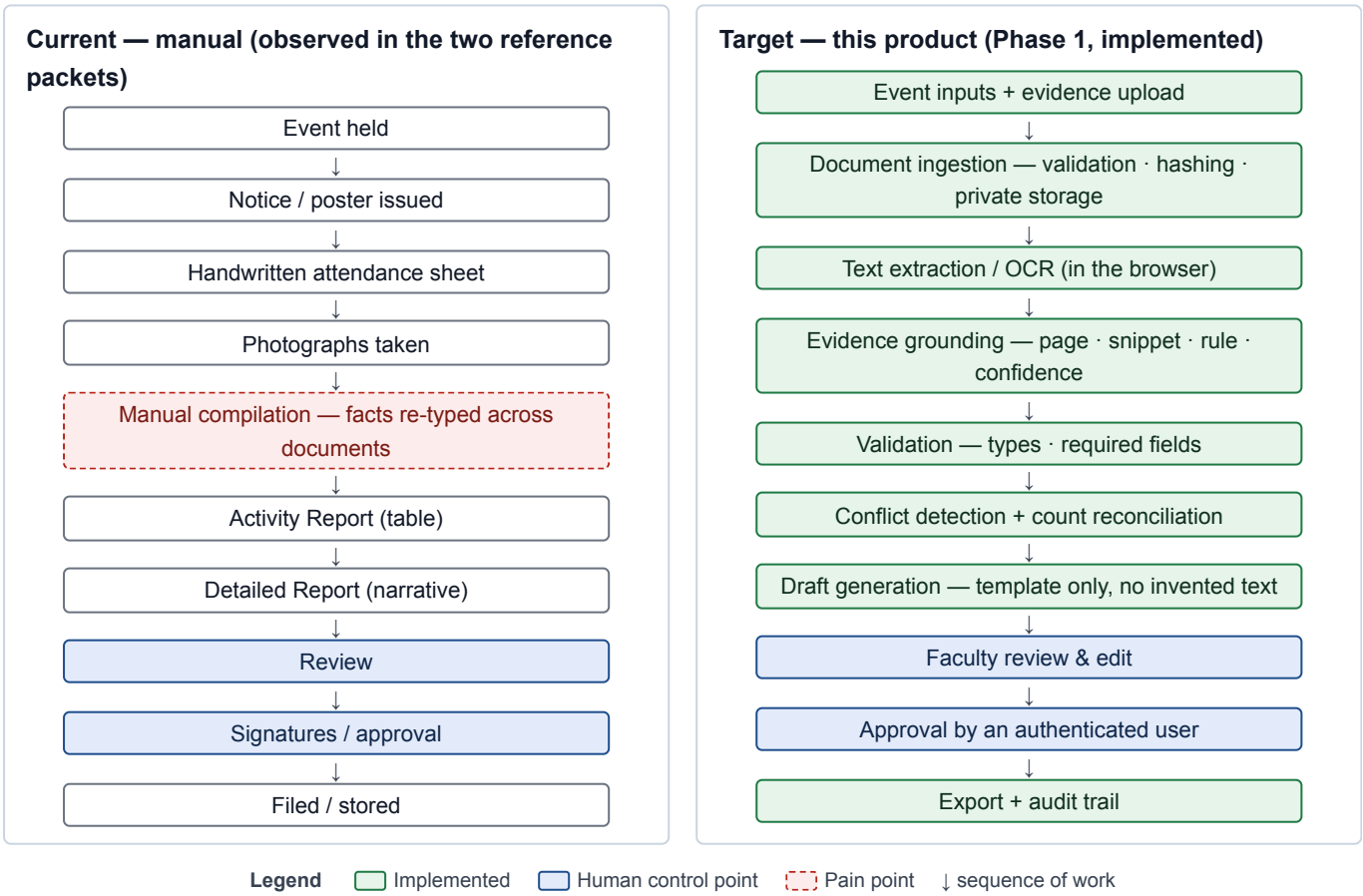
- an **Activity Report** (structured table);
- a **Detailed Report** (narrative with Topic, Event description and Impact Analysis);
- an **Attendance Summary**;
- an auditable **evidence package**.

Every factual value records the document and page it came from. Faculty review and approve the output.

Where the workflow comes from. The pilot department supplied two complete activity packets from session 2024-25. Each packet is one scanned PDF containing, in order: poster, notice, structured activity report, narrative report, handwritten attendance sheet and geotagged photographs. These packets show three things:

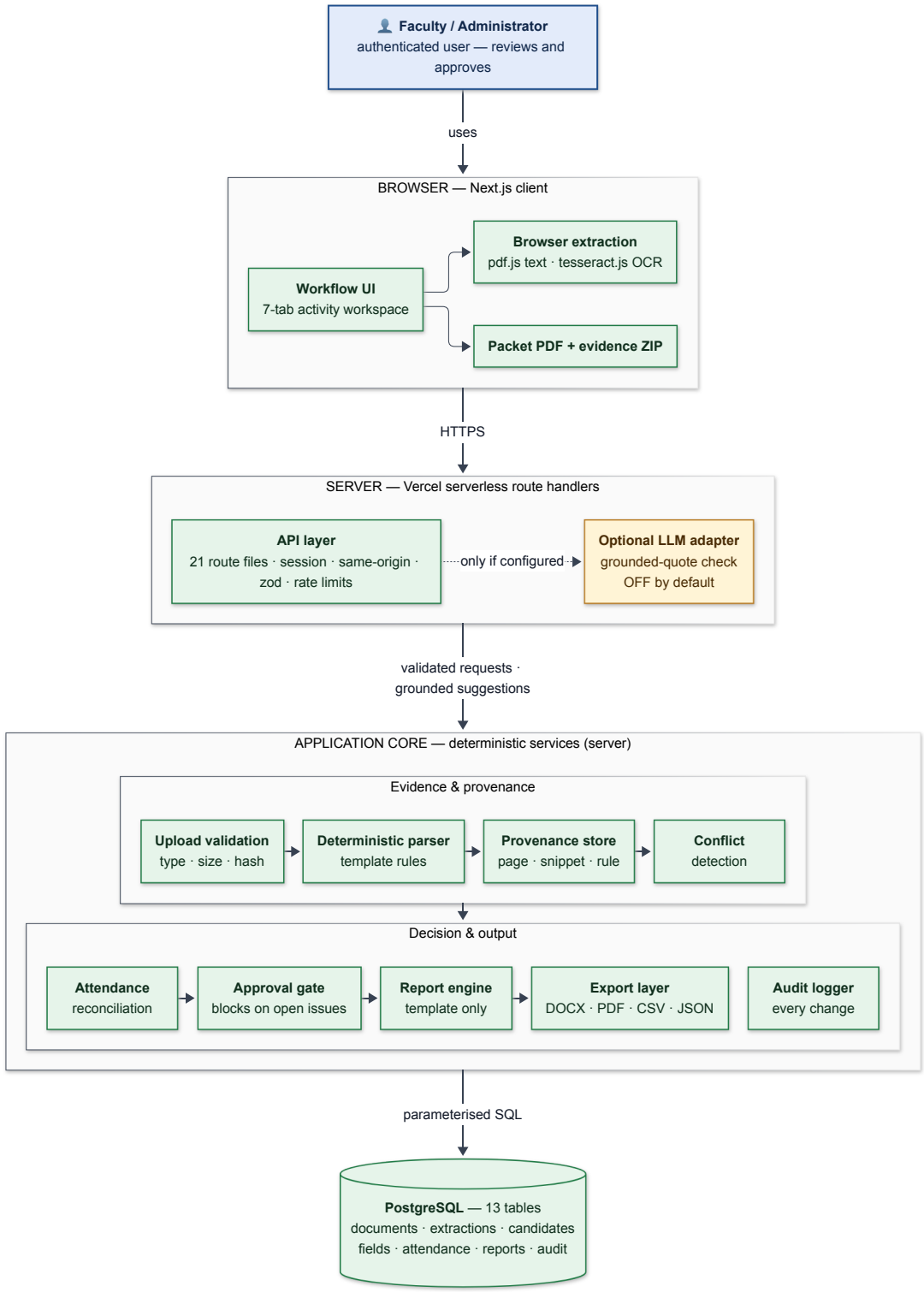
- **Facts are re-typed.** The same title, date, time and participant counts are typed again on four or more documents.
- **Documents disagree.** The posters print the commemorated day (31 Oct 2024). The notices and reports give the activity date (06 Nov 2024). One notice header shows its issue date (05 Nov 2024).
- **Counts are hard to check.** Stated participant counts are difficult to reconcile against handwritten attendance.

A 5-response faculty survey (28 Sep 2026) also points to this workflow. 4 of 5 respondents named activity or event report generation as a recent example of excess effort, and 4 of 5 said "Yes" to testing a prototype.



First concrete workflow: *Activity Documentation Automation* IMPLEMENTED. The AI assists; the faculty member remains the final approver.

2. High-level system architecture — current implementation

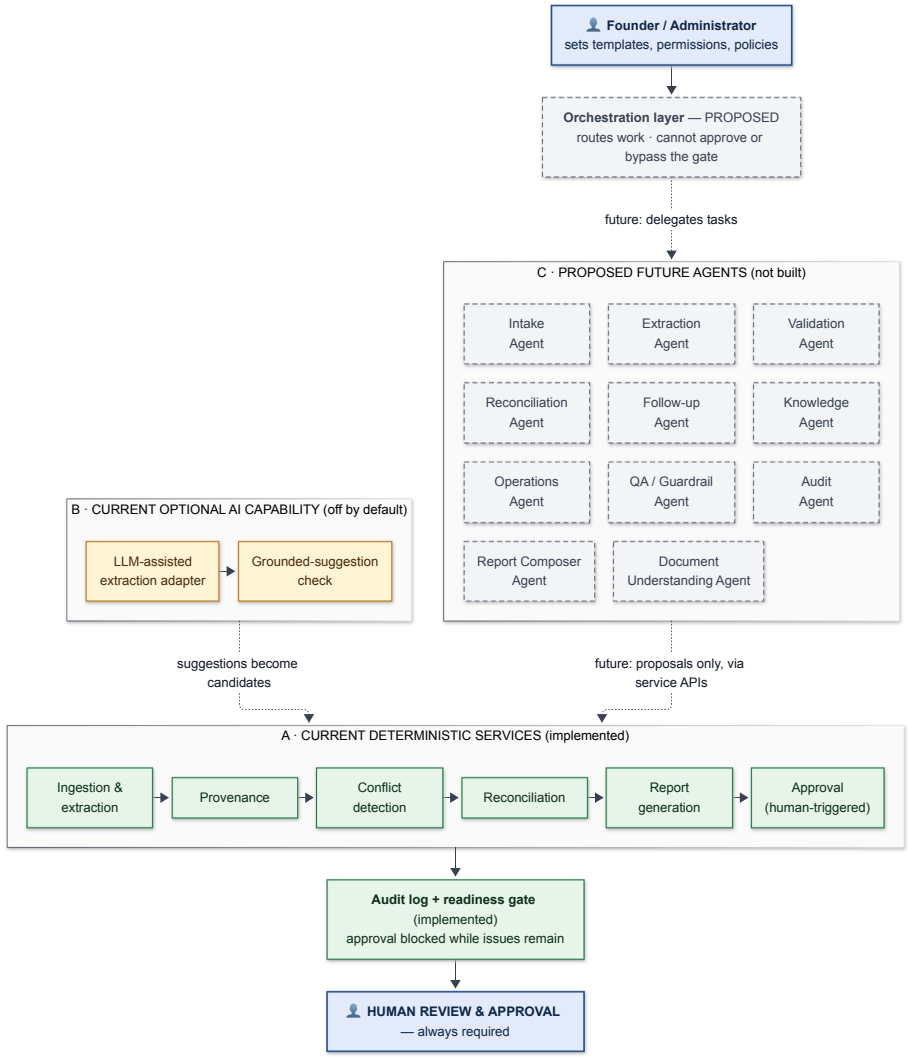


System architecture as implemented (derived from src/ at commit 7119b13)

All boxes are implemented and exercised by automated tests (93 Vitest + 6 Playwright). Code locations are listed in the System Design document.

3. Agent architecture — operating model

The design separates three classes of component. **Deterministic software modules are not presented as AI agents.**



Legend Implemented (verified in code and tests) · Optional — implemented, off by default · Proposed / future — not built · Human control point · → arrows show data / control direction; dotted = optional or proposed

Agent operating model: A current deterministic services, B current optional AI, C proposed future agents

Class	Members	Status
A · Current deterministic services	Document ingestion & extraction · Provenance · Conflict detection · Attendance reconciliation · Report generation · Approval (human-triggered) · Audit	IMPLEMENTED
B · Current optional AI capability	LLM-assisted extraction adapter · grounded-suggestion check (the quote must exist on the cited page)	OPTIONAL · OFF BY DEFAULT
C · Proposed future agents	Orchestrator · Intake · Document Understanding · Extraction · Validation · Reconciliation · Report Composer · QA/Guardrail (quality & compliance) · Audit · Follow-up · Knowledge · Operations	PROPOSED — NOT BUILT

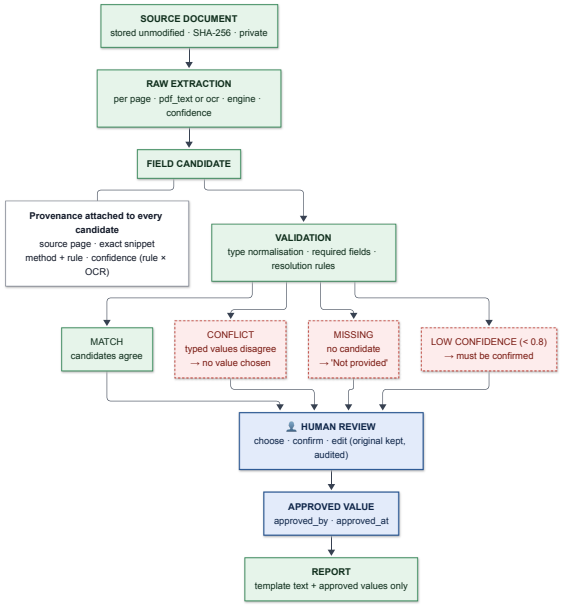
Control rule. In the proposed model, agents may only write *proposals* — candidate values with evidence, draft text or reminder drafts. They do this through the same service layer, validation and audit logging that the web application uses today. The readiness gate and the approval action stay deterministic code and are never agent-callable. Approval always needs an authenticated human.

4. Agent and service cards

Component	Status	Purpose	Inputs → Outputs	Tools / permissions	Changes data? · Approves?	Human checkpoint	Failure behaviour
Ingestion & extraction service	IMPL	Validate, store and read documents	File → stored original + per-page text	`files.ts`, pdf.js, tesseract.js, `parser.ts`; writes `sources`, `source_extractions`, `field_candidates`	Yes: candidates only · No	Reviewer sees every candidate	Upload rejected with a reason; extraction marked `failed`; processing over 15 min treated as failed
Provenance service	IMPL	Keep the evidence for every value	Candidates → field state with source, page, rule, snippet	`activity_fields`, `field_candidates`	Yes: resolved state · No	Edits keep the original value, editor and time	No candidate → `missing`
Conflict detection service	IMPL	Surface disagreement	Candidates → `conflicting` / `extracted`	`fields/resolve.ts`	Yes: status only · No	Reviewer chooses or types a value	Never auto-resolves typed conflicts
Attendance reconciliation	IMPL	Check stated counts against attendance	Stated M/F/T + rows → PASS / FAIL / UNVERIFIED / INSUFFICIENT_DATA	`reconciliation.ts`, `attendance.ts`	Drafts OCR rows (unverified) · No	Verify rows or record an acknowledgement	Mismatch flagged; nothing corrected
Report generation service	IMPL	Build reports from reviewed fields	Fields → versioned report content → DOCX / PDF / CSV / JSON	`reports/build.ts`, `exports/*`	Writes `generated_reports` · No	Preview; regenerate after edits	Missing → "Not provided"; stale reports block approval
Approval service	IMPL	Record the human decision	User action → `APPROVED`, `approved_by`, `approved_at`	`approveActivity`, `readiness.ts`	Yes: status · **Only by a human user**	This *is* the checkpoint	Blocked while any blocking issue exists
LLM-assisted extraction adapter	OPTIONAL	Suggest values with quotes	Page text → grounded candidates (method `llm`, confidence 0.6)	`ai/provider.ts`; writes candidates only	Candidates only · No	Always needs confirmation (below 0.8 confidence)	Unquoted or unsupported values are discarded
Orchestrator Agent	PROPOSED	Plan and route tasks between agents	Activity state → task plan	Read state; call agent tools	No field data · No	Human sees the plan and outcomes	Stops and escalates; never skips the gate
Intake Agent	PROPOSED	Guide faculty through what is needed	Chat or form + uploads → checklist of missing evidence	Read template; create activity draft	Draft only · No	Faculty confirms details	Asks the user; never assumes
Document Understanding Agent	PROPOSED	Classify documents and pages in compiled scans	File → page-kind proposals	Read files; write kind proposals	Proposals only · No	User accepts the page split	Unclassified pages are flagged
Extraction Agent	PROPOSED	OCR and layout-aware field extraction	Pages → candidates with quotes	OCR tools; write candidates	Candidates only · No	Review uncertain fields	Marks the extraction failed or incomplete
Validation Agent	PROPOSED	Explain issues in plain language	Field state → issue explanations	Read-only plus comments	No · No	Reviewer acts on issues	Falls back to the deterministic rules
Reconciliation Agent	PROPOSED	Help verify attendance row by row	Sheet + rows → suggested corrections	Read files; propose row edits	Proposals only · No	Reviewer verifies each row	Leaves rows unverified
Report Composer Agent	PROPOSED	Draft wording from verified fields only	Approved-to-draft fields → narrative draft	Read fields; write draft text	Draft only · No	Faculty edits and accepts	Unsupported sentences rejected by the QA agent
QA / Guardrail Agent	PROPOSED	Check every draft claim against the evidence	Draft + evidence → pass / flag list	Read-only	No · No	Flags shown to the reviewer	Blocks submission for review
Audit Agent	PROPOSED	Summarise the audit trail and detect anomalies	Audit log → review summary	Read-only	No · No	Admin reviews anomalies	Reports only
Follow-up Agent	PROPOSED	Draft reminders for missing evidence or approvals	Pending items → message drafts	Draft messages only	No · No	Human sends each message	Drafts expire
Knowledge Agent	PROPOSED	Cited answers from calendar, timetable, notices	Question + documents → cited answer	Read-only search	No · No	Answers show their sources	"Not found in the provided documents"
Operations Agent	PROPOSED	Track recurring operational tasks across departments	Task register → status digest	Read-only / drafts	No · No	Owner reviews the digest	Reports gaps only

5. Zero-hallucination and trust architecture

> **No unsupported factual value may silently enter an approved report.**



Legend Implemented (verified in code and tests) Human control point Issue / not yet provisioned External / input

→ arrows show data / control direction; dotted = optional or proposed

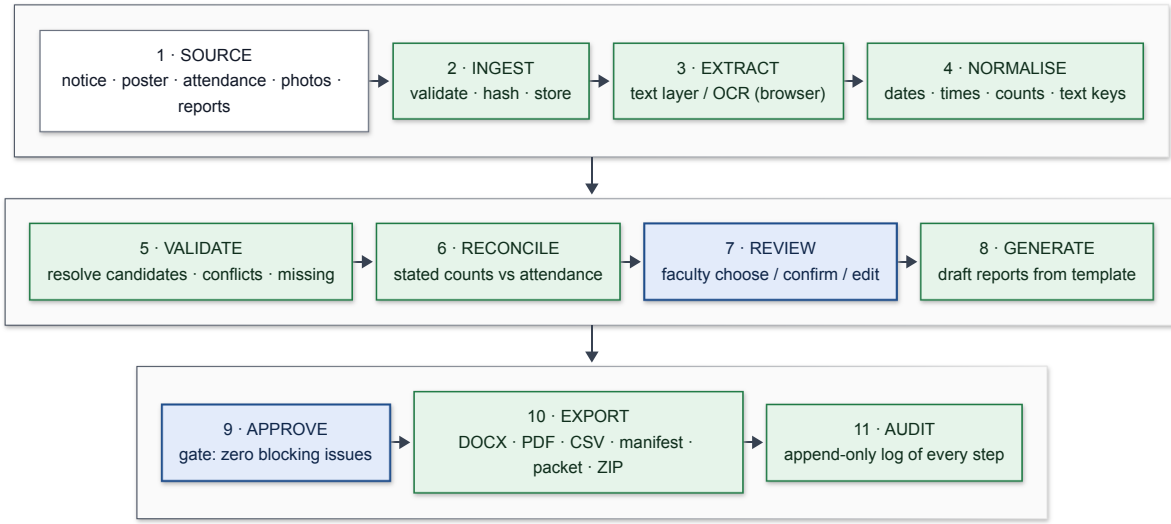
Trust chain from source document to report (all stages implemented)

Rule	How the current code enforces it
Missing ≠ invented	No candidate → status `missing`. Reports print “Not provided”. Required fields block approval.
Conflict ≠ automatically resolved	Date, time and count candidates that disagree → `conflicting` with no value (`fields/resolve.ts`). Only a reviewer's choice or entry resolves it.
OCR output ≠ verified fact	Confidence is the lower of the rule's confidence and the page's OCR confidence. Below 0.8 it must be confirmed. Fuzzy label matches are capped at 0.75. OCR attendance rows are always `verified = false`.
AI suggestion ≠ approved fact	LLM values need a verbatim quote found on the cited page, are stored as method `llm` at confidence 0.6, and need confirmation.
Human approval required	`APPROVED` is set only by an authenticated user's explicit action; the readiness gate must show zero blocking issues.
Source provenance retained	Each candidate stores source, page, rule, snippet and confidence. The JSON manifest exports all of this.
Manual changes are audited	Edits keep the original value, editor and time. `audit_logs` records every write, export and approval.

Evidence from tests on the two real reference packets (local only, not published):

- The poster date (31/10/2024) and the notice and report date (06/11/2024) are **surfaced as a conflict**. No date is printed until the reviewer chooses.
- The notice issue date is stored as a separate field and never used as the activity date.
- Handwritten attendance rows stay **unverified OCR drafts**.
- **Gender is read only from the gender column** and never inferred from names.
- **Approval is blocked** while issues remain.
- The empty “Invited Speaker” cell stays `missing`.

6. Data flow and information lifecycle



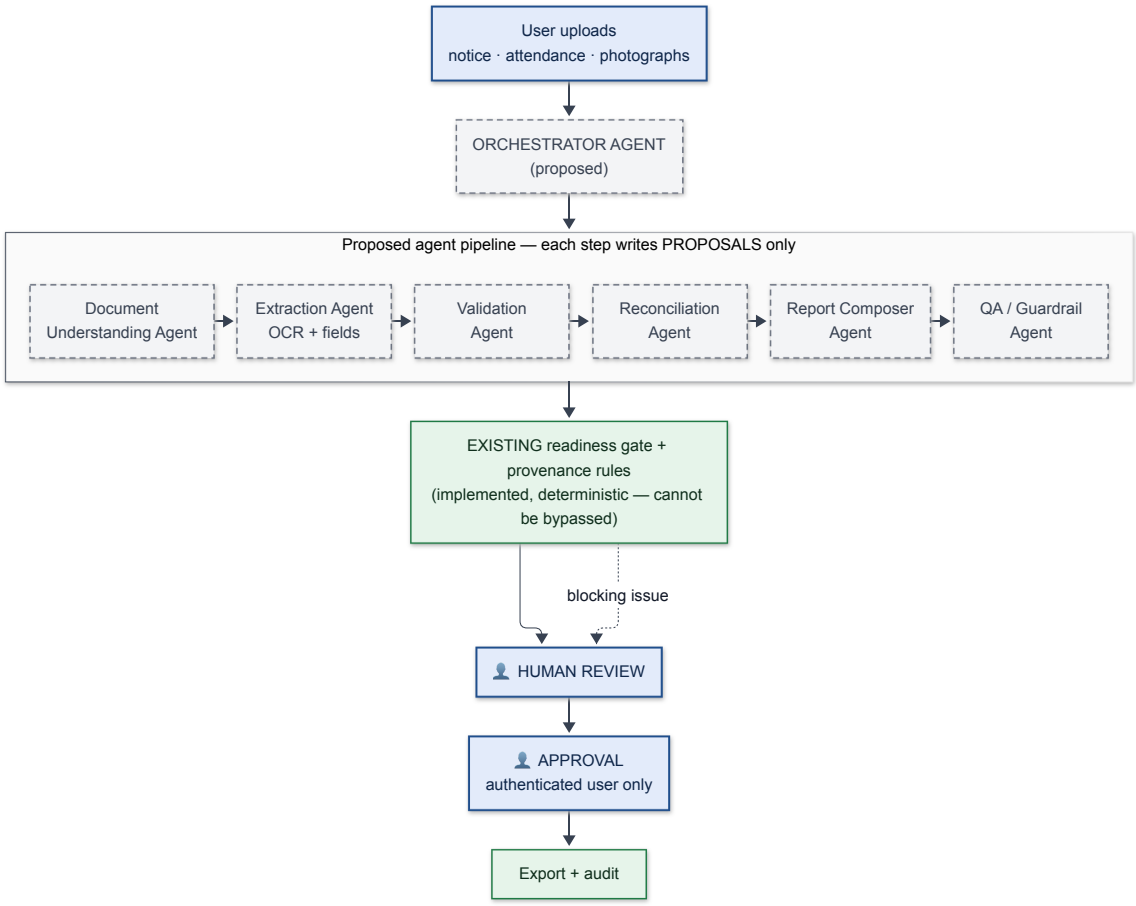
Information lifecycle as implemented

Stage	Input	Output	Owner	Storage	Failure condition
1 Source	Paper or digital documents	Files	Faculty	—	Not available → enter details manually
2 Ingest	File + document kind	Stored original, SHA-256	Upload validation	`sources`, `source_blobs`	Wrong type, over 4 MB, corrupt, duplicate → rejected with a reason
3 Extract	Original file (in the browser)	Per-page text, method, confidence	Browser extractor	`source_extractions`	No readable text → `failed`
4 Normalise	Page text	Typed candidate values	Parser + normaliser	`field_candidates`	Unparseable value → no candidate
5 Validate	Candidates	Field state	Resolution rules	`activity_fields`	Conflict or missing → blocking issue
6 Reconcile	Stated counts + rows	Check results	Reconciliation service	computed	FAIL → blocking unless acknowledged
7 Review	Issues	Choices, confirmations, edits	**Faculty**	`activity_fields`, `audit_logs`	—
8 Generate	Reviewed fields	Versioned report content	Report engine	`generated_reports`	Stale after edits → blocking
9 Approve	Zero blocking issues	`APPROVED` + approver + time	**Faculty**	`activities`	Rejected with the list of issues
10 Export	Latest reports + files	DOCX, PDF, CSV, JSON, packet, ZIP	Export layer	downloads (audited)	Not generated → “Generate the reports first”
11 Audit	Every step	Append-only log	Audit logger	`audit_logs`	—

Ordering note: the brief listed Approve before Generate. In the implementation, reports are generated **before** approval, so the approver signs off on an exact report version. Exports made before approval carry a “DRAFT — not yet approved” footer.

7. Agent orchestration — future architecture

PROPOSED — NOT BUILT This page describes how the proposed agents would cooperate. Nothing on it runs today, apart from the green gate and the human steps.



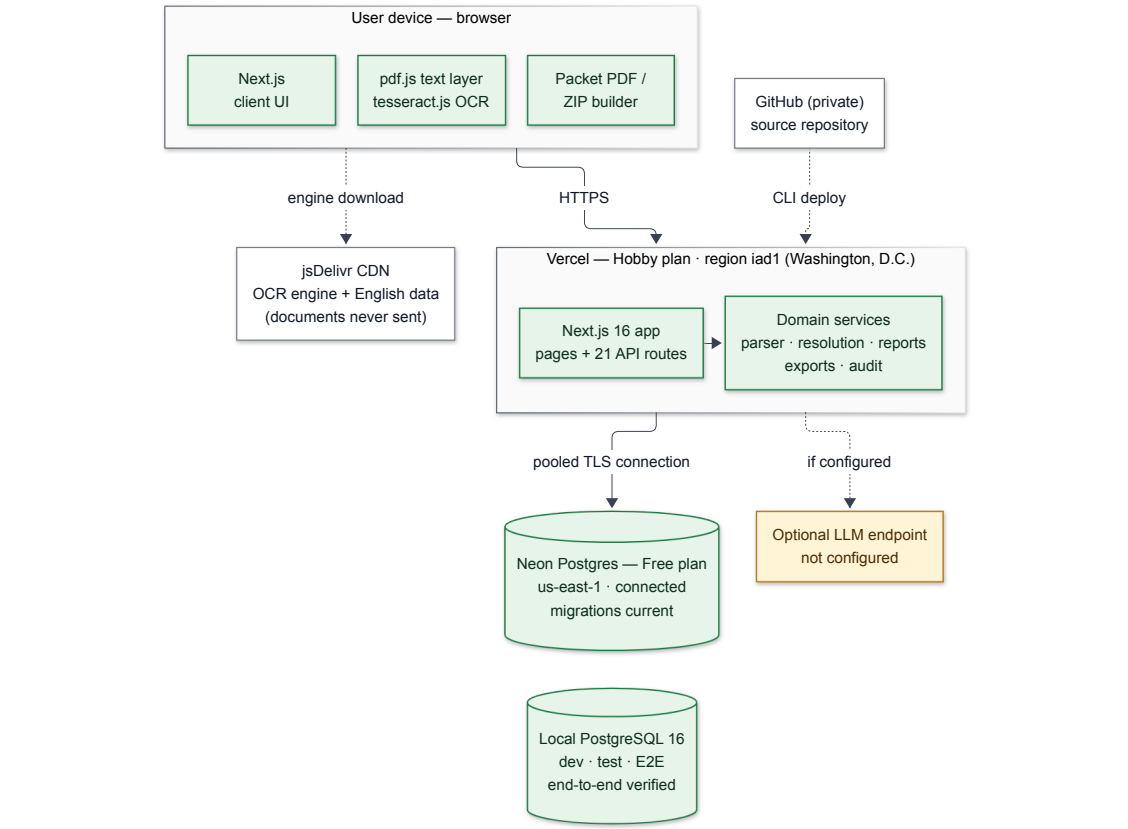
Proposed orchestration: agents produce proposals; the existing deterministic gate and a human approval cannot be bypassed

Orchestration rules (design constraints for any future version):

1. **Proposals, not decisions.** Every agent writes candidate values, draft text or message drafts. Nothing an agent writes is marked `user_entered` or approved.
2. **Same door as humans.** Agents call the existing service functions, so upload validation, type normalisation, access control and audit logging apply unchanged. Every action is logged with the agent id, model and prompt version.
3. **Grounding is mandatory.** Any factual value must cite a source page and a verbatim quote. The current `groundLlmValue` check is the reference implementation.
4. **The gate is code.** The readiness gate (`readiness.ts`) and approval stay deterministic and cannot be called by an agent. The orchestrator cannot skip validation, reconciliation or review.
5. **Escalate, don't guess.** When evidence is missing or contradictory, the agent reports it to the human and stops.
6. **No external action without a human.** Follow-up messages are drafts until a person sends them.

This design is deliberately **not** fully autonomous.

8. Deployment architecture — current state



Deployment topology as verified on 30 Sep 2026

Item	Verified state (30 Sep 2026)
Local end-to-end	**Working.** 93 Vitest tests (unit, Postgres integration, permissions, exports, two real reference cases) and 6 Playwright browser tests against the production build pass, including real in-browser OCR.
Source code	Pushed to a **private** GitHub repository.
Vercel deployment	**Live** (Hobby plan, region `iad1`, next to the database): `college-operations-copilot.vercel.app`.
Production database	**Neon Postgres, Free plan** (`us-east-1`), connected through the Vercel Marketplace. Migrations current; the schema was verified object by object against a reference database.
Deployments	Made with the Vercel CLI. An earlier `TEAM_ACCESS_REQUIRED` block was resolved by linking the owner's GitHub login to the Vercel account.
Signed-in flow on the live URL	**Verified.** Production smoke test 24/24 (login, upload, OCR, provenance, date conflict, attendance, reports, approval, DOCX/PDF export, access control, logout), cross-checked in the production database. Synthetic documents only.

Current production topology:

- **Browser.** UI, pdf.js text extraction, tesseract.js OCR (engine downloaded from jsDelivr), packet and ZIP assembly.
- **Vercel Node functions.** Pages, API routes and domain services.
- **Postgres.** All records, including original files.
- **Optional LLM endpoint.** Only if configured.

No paid service is used.

9. Implemented vs future

Feature	Status	Evidence
Authentication (admin-provisioned accounts, hashed sessions, rate limits)	IMPLEMENTED	`auth-core.ts`, permissions tests, E2E
Activity workflow (7-tab workspace)	IMPLEMENTED	`activities/[id]/*`, E2E
File upload (type sniffing, 4 MB cap, corrupt/duplicate rejection)	IMPLEMENTED	`files.ts`, unit + E2E
Browser extraction (text layer, geometric line rebuild)	IMPLEMENTED	`browser.ts`, `layout.ts`
OCR (tesseract.js, English, render width chosen by measurement)	IMPLEMENTED	browser OCR E2E on a real scan
Provenance (page, rule, snippet, confidence per candidate)	IMPLEMENTED	integration tests
Conflict detection	IMPLEMENTED	unit, integration, reference cases
Attendance review and reconciliation	IMPLEMENTED	unit, integration, E2E
Activity Report · Detailed Report · Attendance Summary	IMPLEMENTED	`reports/build.ts`, export tests
Approval workflow and readiness gate	IMPLEMENTED	integration, E2E
Audit trail	IMPLEMENTED	integration, E2E
Exports: DOCX, PDF, CSV, JSON manifest, packet PDF, ZIP	IMPLEMENTED	export tests, E2E downloads
Configurable, versioned templates	IMPLEMENTED	`templates/*`
Optional LLM adapter with grounding check	IMPLEMENTED · OFF BY DEFAULT	unit test with a mocked endpoint; never run against a live model
Knowledge Agent (calendar, timetable, notices)	FUTURE	—
Student-facing assistant	FUTURE	—
Follow-up Agent	FUTURE	—
Operations Agent	FUTURE	—
Multi-agent orchestration	FUTURE	—
Institution-wide / multi-department deployment	FUTURE	—
Hosted production deployment (Vercel + Neon, free tiers)	LIVE — VERIFIED	production smoke test 24/24, §8

10. Why this architecture

1. **Evidence-first.** A value exists only because a document page says it or a person entered it.
2. **Human-in-the-loop.** Software proposes; faculty choose, confirm, edit and approve.
3. **Provenance-first.** Source, page, rule, snippet and confidence travel with every value, into the export manifest.
4. **Deterministic validation.** Conflicts, missing data, confidence thresholds and count reconciliation are plain code with tests, not model judgement.
5. **Configurable templates.** Institution wording and layout are data (JSON). A test asserts that no reference event name appears in the source code.
6. **Separation of extraction from approval.** Extraction can only produce candidates; approval is a separate, human-only action.
7. **AI optionality.** The system runs fully without a model. A model, if added, is held to the same grounding rule.
8. **Auditability.** Append-only audit log, versioned reports, and a manifest per activity.
9. **Privacy by design.** OCR runs in the browser, files are stored privately, real reference documents are never committed or deployed, and gender is never inferred from names.
10. **Incremental agentization.** Agents can be added one at a time behind the existing gate without changing the trust model.

Originality and Scope

This project presents a purpose-built architecture for automating college activity documentation through evidence-grounded extraction, provenance tracking, conflict reconciliation, human approval and report generation. The architecture is original to this project in its specific integration and implementation. This document does not claim that individual technologies or concepts are globally unprecedented.

This architecture is purpose-built for the targeted college documentation workflow, combining evidence-grounded extraction, provenance, conflict reconciliation, human approval and report generation in one operating model.

Novelty claim requires separate prior-art/competitive research. None has been performed for this document.

11. Roadmap

All phases after Phase 1 are **future roadmap**. Nothing in them is built.

Phase	Scope	Status
Phase 1 — Activity Documentation Automation	Evidence upload, extraction with provenance, conflict and attendance checks, three reports, approval, exports, audit	CURRENT — IMPLEMENTED (live, verified)
Phase 2 — College Knowledge Agent	Question answering over the timetable, academic calendar and notices, every answer citing page sources. The two sample documents already in the pilot folder (calendar, timetable) motivate this phase.	FUTURE
Phase 3 — Workflow & Follow-up Agents	Reminders for missing evidence and pending approvals (drafts, human-sent); Intake and Document Understanding agents; per-page classification of compiled scans	FUTURE
Phase 4 — Student-facing assistant	Read-only answers for students from approved institutional sources	FUTURE
Phase 5 — Cross-department operations platform	Multiple departments and committees, shared oversight, institution-wide deployment	FUTURE

Before any agent phase begins:

- The hosted database is attached and the Vercel access issue is fixed.
- The pilot is run with real faculty users, and extraction accuracy is measured on more than the two reference packets.
- Agent permissions (the “proposals only” rule) are implemented as a service principal with an allow-listed toolset.